

# Project Title: AI-Powered Industrial Quality Control (QC) System

## Project Objective

You will build an end-to-end Computer Vision application using **Teachable Machine**, **Python**, and **Streamlit**. Your system will simulate an automated conveyor belt inspection assistant that identifies whether a specific fruit category is **Fresh** or **Rotten**, deploying real-time automated visual/audio alerts if a defective product is detected.

## Phase 1: Dataset Architecture & Data Collection

Before writing any code, you must design your data pipeline. Answer and fulfill the following requirements:

1. **Target Fruit Selection:** Which single fruit category are you tracking for your industrial QC simulation? (e.g., Bananas, Apples, Tomatoes).
2. **Environmental Controls:** To ensure the model learns the condition of the fruit rather than the background, how will you keep your lighting, camera angles, and backgrounds consistent across both classes?
3. **Dataset Diversity Target:** You are required to collect at least **50–100 custom images** per class using your phone or webcam. Fill out your dataset target map below:
  - **Class 1 (Fresh):** Number of samples: \_\_\_\_\_ | Varietals used (e.g., green, bright yellow): \_\_\_\_\_
  - **Class 2 (Rotten):** Number of samples: \_\_\_\_\_ | Stages of decay captured (e.g., brown spots, mold): \_\_\_\_\_

## Phase 2: Technical & Code Requirements

Your final script must be adapted from your foundational template and include the following precise logic boundaries:

1. **Python 3.12 & Legacy Compatibility:** Your script must use the dynamic path utility `os.path.join` and the `tf_keras` legacy library to ensure it initializes smoothly without crashing on version mismatches.
2. **Channel Matrix Preprocessing Alignment:** The app must be robust enough to handle any image variation a user uploads. You must write conditional safety statements to handle:
  - Grayscale images (converting 2D shapes to 3-channel RGB).
  - Alpha transparency layers (stripping 4D PNG profiles to 3D spaces).
3. **Threshold Logic & Alert Mechanisms (Critical Requirement):** Instead of using simple comparative logic, you must implement a safety guard gate loop in **Step 6**.
  - If **Rotten** confidence is **greater than 75% ( $>0.75$ )**, the UI must trigger a distinct red danger notification using `st.error()` or `st.alchemy`.
  - **Bonus Challenge:** Integrate an audio alert. Research how to use a hidden HTML `<audio>` tag or native Streamlit audio components to trigger an alarm sound when a rotten item passes the threshold.

# Student Reflection & Diagnostics Questionnaire

Students must complete and submit these questions along with their working application link/code repository.

## Part A: Algorithmic Mechanics

1. **Dimension Constraints:** Why must the uploaded user image go through `ImageOps.fit(image, (224, 224))` before reaching `model.predict()`? What happens to the matrix math if a raw  $1080 \times 1920$  pixel phone photo is passed directly instead?
2. **Normalization Vector Calculation:** Explain the mathematical purpose of this line in your script:

```
normalized_img_array = (img_array.astype(np.float32) / 127.5) - 1
```

Why map the scale to ranges between  $[-1.0, 1.0]$  instead of keeping standard RGB raw integers  $[0, 255]$ ?

## Part B: Industrial Quality Control Deployment

3. **Threshold Optimization:** You set your industrial decay alert threshold to 75%.
  - What are the real-world consequences if you set the threshold too low (e.g., **40%**)? (*Hint: Think about False Positives wasting good food*).
  - What happens if you set it too high (e.g., **98%**)? (*Hint: Think about False Negatives letting rotten food get shipped to stores*).
4. **Edge Case Analysis:** Test your completed application with a "trick" image—such as a plastic toy fruit, a cartoon drawing of your fruit, or a completely blank white screen.
  - What did your model predict?
  - Based on this result, what are the current limitations of your AI agent, and how would you fix it in a production setting?

## Project Report Submission Checklist

- ☐ Working app.py script cleanly commented matching the 6-step architecture framework.
- ☐ Valid compiled keras\_model.h5 file sitting in the root project folder directory.
- ☐ Standalone requirements.txt file listing packages (streamlit, numpy, pillow, tensorflow, tf\_keras).
- ☐ Completed copy of this Requirement and Questionnaire worksheet.